# 图书管理系统

一个全面的图书管理系统，包含图书借阅、捐赠跟踪和活动管理功能。

## 技术栈

### 核心

- **Go 1.23+** - 主要编程语言
- **Gin** - HTTP web框架
- **GORM** - 数据库操作ORM库

### 数据库

- **MySQL 5.7+** - 主要关系型数据库
- **Redis 6+** - 性能优化缓存层

### 基础设施

- **Docker** - 容器化
- **Docker Compose** - 多容器编排
- **Wire** - 编译时依赖注入

### 文档

- **Swagger** - API文档生成
- **Makefile** - 构建自动化

### 测试

- **Go Test** - 单元测试框架
- **GitHub Actions** - CI/CD流水线

## 架构概述

系统采用清晰架构模式，关注点分离明确：

```
├── cmd/            # 应用入口
├── configs/        # 配置文件
├── docs/           # API文档
├── internal/       # 核心应用逻辑
|   ├── controller/ # HTTP处理器
|   ├── service/    # 业务逻辑
|   ├── repository/ # 数据访问层
|   |   ├── cache/  # Redis缓存
|   |   ├── dao/    # 数据库操作
|   |   └── repo/   # 仓库接口
|   ├── ioc/        # 依赖注入
|   └── middleware/ # HTTP中间件
```

关键架构特性：

- 使用Wire进行依赖注入
- 分层架构(Controller-Service-Repository)
- 缓存层提升性能
- Swagger API文档
- 容器化部署

# 前置要求

- Go 1.23+
- MySQL 5.7+
- Redis 6+
- Docker (可选)

# 配置

创建`configs/config.yaml`文件，结构如下：

```yaml
db:
  addr: "mysql_user:mysql_password@tcp(localhost:3306)/bm?charset=utf8mb4&parseTime=True&loc=Local"
cache:
  addr: "localhost:6379"
users:
  - username: "admin"
    password: "admin123"
```

# 本地开发

1. 安装依赖：

```
go mod download
```

2. 生成Swagger文档：

```
make swagger
```

3. 运行应用：

```
go run cmd/main.go -conf configs/config.yaml
```

# Docker部署

1. 构建Docker镜像：

```
docker build -t bm .
```

2. 启动服务：

```
docker-compose up -d
```

# 状态码

## 图书状态

- `waiting_return`: 图书已借出待归还
- `returned`: 图书已归还
- `overdue`: 图书逾期未还

## 库存状态

- `adequate`: 库存充足
- `early_warning`: 库存预警
- `shortage`: 库存不足

## 图书分类

- `children_story`: 儿童故事书
- `science_knowledge`: 科普知识书
- `art_enlightenment`: 艺术启蒙书

## 活动状态

- `pending`: 活动待开始
- `ongoing`: 活动进行中
- `ended`: 活动已结束

## 用户状态

- `normal`: 用户正常
- `overdue`: 用户有逾期图书
- `freeze`: 用户被冻结

# API文档

## 认证

- `POST /api/auth/login` - 用户登录
  - 请求:

    ```
    {
      "username": "admin",
      "password": "admin123"
    }
    ```

  - 响应:

```
{
  "code": 200,
  "message": "success",
  "data": {
    "token": "JWT_TOKEN"
  }
}
```

## 图书管理

- `GET /api/books` - 获取所有图书
  - 响应:

```
{
  "code": 200,
  "message": "success",
  "data": [
    {
      "id": 1,
      "title": "图书标题",
      "author": "作者名称",
      "status": "waiting_return"
    }
  ]
}
```

- `POST /api/books/borrow` - 借阅图书
  - 请求:

```
{
  "book_id": 1,
  "user_id": 1
}
```

  - 响应:

```
{
  "code": 200,
  "message": "success",
  "data": {
    "borrow_id": 1,
    "due_date": "2025-05-15"
  }
}
```

## 活动管理

- `GET /api/activities` - 获取所有活动
  - 响应:

```
{
  "code": 200,
  "message": "success",
  "data": [
    {
      "id": 1,
      "name": "阅读活动",
      "status": "ongoing"
    }
  ]
}
```

## 捐赠管理

- `POST /api/donations` - 捐赠图书
  - 请求：

```
{
  "book_id": 1,
  "donor_id": 1,
  "quantity": 2
}
```

  - 响应：

```
{
  "code": 200,
  "message": "success",
  "data": {
    "donation_id": 1,
    "status": "pending"
  }
}
```

更详细的API文档包含所有端点和参数，可通过Swagger UI访问：

```
http://localhost:8989/swagger/index.html

```